Universidad Diego Portales
ESCUELA DE INFORMÁTICA &
TELECOMUNICACIONES

ESTRUCTURAS DE DATOS &
ANÁLISIS DE ALGORITMOS

Laboratorio 1: Análisis empírico y teórico de algoritmos

Autores:
Renato Parra
Matias Herrera

Sección 5

Profesor:
Marcos Fantoval

Fecha de entrega
22-03-2026

Índice

1. Introducción **2**

2. Etapa 1: Análisis empírico **2**
2.1. (I) Estructura del software 2
2.2. (II) Desafíos enfrentados 3
2.3. (III) Análisis empírico 3

3. Etapa 2: Resultados en Excel **4**

4. Etapa 3: Análisis teórico **5**
4.1. Método 1: Dos Punteros (`cuentaPares`) 6
4.2. Método 2: Búsqueda Binaria (`cuentaPares2`) 6
4.3. Método 3: Fuerza Bruta (`cuentaPares3`) 7

5. Etapa 4: Comparación teoría vs. experimento **7**

6. Conclusiones **8**

1. Introducción

En este laboratorio, realizamos un estudio empírico y teórico del tiempo de ejecución de tres algoritmos diferentes que resuelven el mismo problema: contar cuántos pares de números dentro de un arreglo tienen una diferencia exacta igual a un valor k.

El objetivo principal de este trabajo es medir los tiempos de ejecución en Java probando distintos tamaños de entrada (n), para luego comparar si el comportamiento empírico de los algoritmos coincide con su complejidad teórica (Big-O).

Junto con este informe en PDF, se hace entrega de los siguientes archivos correspondientes a las distintas etapas del proyecto:

- **Código fuente:** El archivo `main.java` que usamos para generar los datos y tomar los tiempos.
- **Archivos CSV:** Seis archivos (desde `resultados_n32.csv` hasta `resultados_n1024.csv`) con los tiempos obtenidos para cada instancia.
- **Planilla Excel:** El archivo `analisis.xlsx` con los datos de los archivos `csv`, gráficos log-log y el resumen de la métrica de duplicación.

2. Etapa 1: Análisis empírico

2.1. (I) Estructura del software

Nuestro programa está compuesto por dos clases principales. Primero está `Result.java`, que nos fue entregada e incluye los tres métodos a evaluar (`cuentaPares`, `cuentaPares2` y `cuentaPares3`).

Por otro lado está nuestra clase `main.java`, que controla el experimento, el cual se basó fielmente en el modelo arquitectónico de la clase `DoublingRatio`, escalando el tamaño de la instancia en potencias de dos. Para cumplir con la estrategia de duplicación, el programa prueba los tamaños de n desde 32 hasta 1024. Los números aleatorios los generamos usando `StdRandom.uniform()`. Para asegurar la reproducibilidad de los datos, seteamos una semilla fija usando `StdRandom.setSeed(1000 + t)` al inicio del programa. El tiempo de ejecución específico de cada método lo medimos envolviendo el llamado a la función con `StopwatchCPU`, y finalmente guardamos todo en archivos usando la clase `Out`, siendo los resultados agrupados en archivos CSV, categorizados por el volumen de entrada (n). Para que la medición sea justa, clonamos la lista original antes de pasársela a cada método, así evitamos que el ordenamiento (`sort`) de un método afecte al siguiente.

2.2. (II) Desafíos enfrentados

Lo más difícil al principio fue configurar el entorno de trabajo (IDE) para que reconociera las clases de Princeton. Ya que nos daba error al intentar compilar porque no encontraba `StdRandom` ni `StopwatchCPU`. Tuvimos que investigar cómo agregar correctamente las dependencias de `algs4` a la carpeta de nuestro proyecto.

A nivel de código, el mayor desafío fue generar las instancias cumpliendo la regla de que los números no podían repetirse. Si usábamos un arreglo normal, revisar si el número ya existía iba a ser muy lento. Lo solucionamos usando la estructura `HashSet<Integer>`: generábamos números con un ciclo `while` y los metíamos al Set hasta que su tamaño fuera exactamente n, garantizando así elementos únicos de forma eficiente.

2.3. (III) Análisis empírico

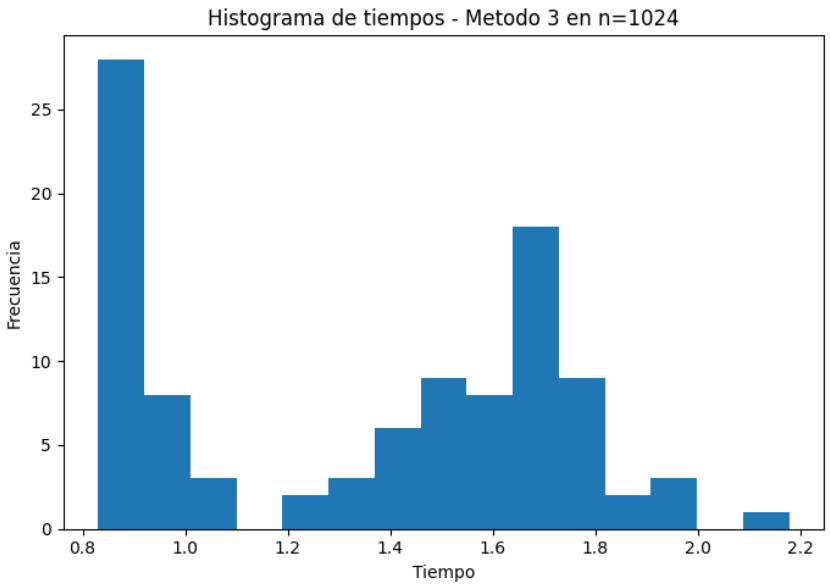

Figura 1: Histograma de los tiempos del Método 3 (fuerza bruta) para $n = 1024$.

Hicimos pruebas con 100 instancias por cada tamaño de $n \in \{32, 64, 128, 256, 512, 1024\}$. Al revisar los promedios y los histogramas, queda claro cómo se comportan los distintos algoritmos. Para los Métodos 1 y 2, los tiempos se mantienen casi en cero incluso cuando llegamos a 1024. En cambio, en el Método 3 (fuerza bruta), los tiempos se empiezan a disparar rápidamente. Con la estrategia de duplicación confirmamos que mientras los dos primeros métodos casi no sienten el aumento de datos, el Método 3 multiplica su tiempo por 4 cada vez que duplicamos n.

3. Etapa 2: Resultados en Excel

n	Promedio1	Promedio2	Promedio3	Ratio1	Log2(ratio1)	Ratio2	Log2(ratio2)	Ratio3	Log2(ratio3)
32	0.0766	0.057333	0.178383						
64	0.08309	0.058978	0.155967	1.084735	0.117343	1.028696	0.040817	0.874337	-0.19374
128	0.072085	0.060946	0.029424	0.867546	-0.20499	1.033371	0.047358	0.188657	-2.40616
256	0.114662	0.119507	0.102975	1.590653	0.66962	1.960864	0.971489	3.499688	1.807226
512	0.090322	0.125521	0.361108	0.787721	-0.34424	1.050328	0.070841	3.506744	1.810132
1024	0.159811	0.205076	1.340908	1.769355	0.823224	1.633798	0.70823	3.713312	1.892706

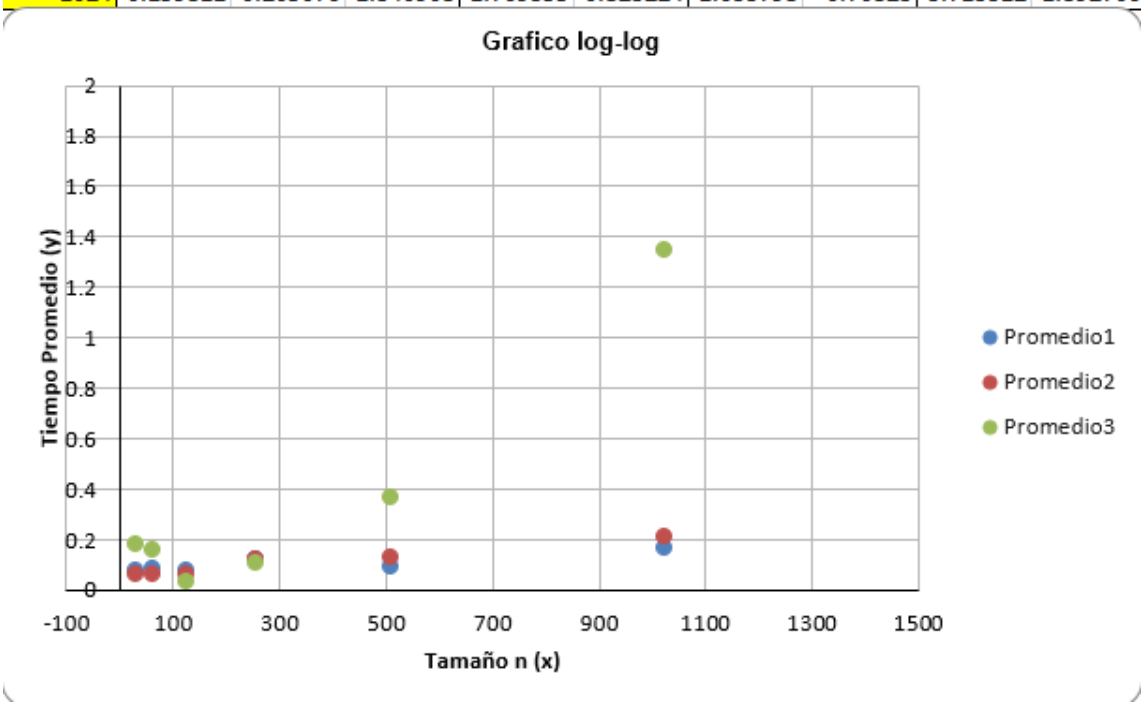

Figura 2: Resumen de promedios, ratios y límite asintótico desde la planilla Excel.

En la planilla `analisis.xlsx` ordenamos todos los datos recopilados. Las primeras seis pestañas corresponden a cada tamaño n evaluado. Donde aparecen los 100 tiempos de cada método, calculamos los mínimos, máximos, promedios y desviación estándar de cada método, además de armar los histogramas.

En la pestaña resumen juntamos los promedios, tambien se calcularon explícitamente los ratios de crecimiento T (2n)/T (n) de cada método, junto a los logaritmo base 2 (log2(ratio)).

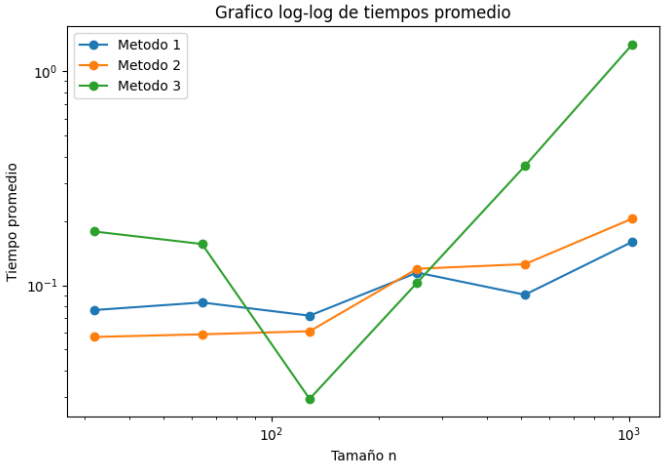

Figura 3: Gráfico log-log de los tiempos promedios.

El gráfico log-log nos muestra de forma visual este crecimiento. La línea verde (Método 3) tiene una pendiente muy marcada y recta, lo que es clásico de una función polinomial de grado mayor (cuadrática). Las líneas azul y naranja (Métodos 1 y 2) van casi pegadas al fondo, confirmando que crecen mucho más lento.

4. Etapa 3: Análisis teórico

Aquí hacemos el cálculo manual del Big-O para justificar los resultados que vimos en el Excel, enfocándonos en lo que pasa cuando $n \to \infty$.

4.1. Método 1: Dos Punteros (`cuentaPares`)

```java
public static int cuentaPares(int pN, List<Integer> pValores, int pK) {
  int izq = 0;
  int der = 0;
  int res = 0;

  pValores.sort(null);  O(NlogN)
  while (der < pValores.size()) {  O(N)
    if (pValores.get(izq) + pK > pValores.get(der)) {
      der += 1;
    } else if (pValores.get(izq) + pK == pValores.get(der)) {
      res += 1;
      izq += 1;
      der += 1;
    } else if (pValores.get(izq) + pK < pValores.get(der)) {
      izq += 1;
    }
  }
  return res;  O(NlogN) + O(N) = O(NlogN
}
```

Figura 4: Código del método 1.

Primero, la función hace un `pValores.sort(null)`, que en Java toma un tiempo de $O(n \log n)$. Después, entra a un ciclo `while` que usa dos punteros (`izq` y `der`). La clave aquí es que los punteros solo avanzan hacia adelante a través del arreglo y nunca retroceden. Entre los dos punteros recorren el arreglo como máximo 2 veces, lo que significa que este ciclo toma un tiempo lineal $O(n)$. Al sumar ambas partes, el ordenamiento es el término que pesa más. Por lo tanto, la complejidad teórica final es $O(n \log n)$.

4.2. Método 2: Búsqueda Binaria (`cuentaPares2`)

```java
public static int cuentaPares2(int pN, List<Integer> pValores, int pK) {
  int res = 0;
  pValores.sort(null);  O(NlogN)
  for (int i = 0; i < pValores.size(); i++) {  O(N)
    if (Collections.binarySearch(pValores, pValores.get(i) + pK) >= 0) {  O(logN)
      res += 1;
    }
  }
  return res;  O(NlogN) + O(N) + O(logN) = O(NlogN)
}
```

Figura 5: Código del método 2.

Al igual que el método anterior, parte ordenando la lista con `sort`, lo que cuesta $O(n \log n)$. Luego hace un ciclo `for` que pasa por todos los elementos de la lista

(n iteraciones). Adentro del `for`, usa `Collections.binarySearch` para buscar el par correspondiente, lo cual toma $O(\log n)$. Como la búsqueda binaria se ejecuta n veces, ese bloque completo nos da un tiempo de $n \cdot O(\log n) = O(n \log n)$. Sumando el ordenamiento y las búsquedas, ignoramos las constantes y la complejidad se mantiene en $O(n \log n)$.

4.3. Método 3: Fuerza Bruta (`cuentaPares3`)

```
public static int cuentaPares3(int pN, List<Integer> pValores, int pK) {
  int res = 0;
  for (int i = 0; i < pValores.size(); i++) {
    for (int j = i + 1; j < pValores.size(); j++) {
      if (Math.abs(pValores.get(i) - pValores.get(j)) == pK) {
        res += 1;
      }
    }
  }
  return res;
}
```

O(N'2)

Figura 6: Código del método 3.

Este método no ordena la lista. Simplemente usa dos ciclos `for` anidados para comparar todos los elementos contra todos. El `for` de afuera (con la variable i) da n vueltas. El `for` de adentro (con la variable j) parte desde $i + 1$, revisando lo que queda del arreglo. Esto hace que las iteraciones sean una suma aritmética ($n-1+n-2+\cdots+1$), lo que matemáticamente da $\frac{n(n-1)}{2}$. Como en Big-O ignoramos las divisiones por 2 y los términos menores (n), nos quedamos solo con la parte que domina. Su complejidad es $O(n^2)$.

5. Etapa 4: Comparación teoría vs. experimento

Para ver si la teoría se cumple en la práctica, cruzamos los Big-O que dedujimos con la tabla de logaritmos que sacamos del programa.

Tamaño n	**Log2(Ratio) M1**	**Log2(Ratio) M2**	**Log2(Ratio) M3**
64	0.117	0.040	-0.193
128	-0.204	0.047	-2.406
256	0.669	0.971	1.807
512	-0.344	0.070	1.810
1024	0.823	0.708	**1.892**

Cuadro 1: Valores experimentales de $\log_2(ratio)$ por tamaño de entrada.

Los resultados del Método 3 (fuerza bruta) calzan perfecto con la teoría. A medida que duplicamos el tamaño hasta llegar a $n = 1024$, vemos que el $\log_2$(ratio) empieza a estabilizarse y converge a **1.892**. Este valor empírico se acerca mucho al exponente 2 teórico que predice el comportamiento $O(n^2)$.

Por el contrario, los valores de los métodos 1 y 2 fluctúan bastante, arrojando valores muy cercanos a cero e incluso negativos. Aunque teóricamente son $O(n \log n)$, los tiempos de ejecución en nuestro PC para $n = 1024$ fueron de milisegundos muy bajos (casi 0). Cuando el tiempo que se demora el algoritmo es tan pequeño, las constantes ocultas de la máquina, los procesos de fondo del sistema operativo y los tiempos propios de la JVM afectan la medición. Aun así, esta gran diferencia confirma lo que dice la teoría: tener un buen algoritmo hace que el programa corra mucho más rápido.

6. Conclusiones

En este laboratorio, se demostró de manera tangible el impacto crítico que posee el diseño de un algoritmo sobre su rendimiento computacional. Frente al mismo problema lógico, comprobamos que la técnica de fuerza bruta (Método 3) posee una escalabilidad insostenible debido a su complejidad cuadrática $O(n^2)$; basta con ingresar un par de miles de datos para que el programa se quede colgado tratando de evaluar todas las combinaciones. Por el contrario, los enfoques que preprocesan la entrada mediante ordenamiento y utilizan punteros o búsqueda binaria (Métodos 1 y 2) reducen drásticamente la complejidad a $O(n \log n)$, logrando resolver el mismo desafío casi al instante.

Finalmente, aprendimos a montar un entorno de experimentación riguroso dentro del IDE y a coordinar nuestro código mediante GitHub. Comprendimos que perfilar el rendimiento computacional exige mucho más que iniciar un cronómetro: requiere aislar la ejecución, clonar los arreglos para no contaminar las muestras y utilizar semillas para asegurar la reproducibilidad del experimento. El contraste entre teoría y práctica nos demostró que el Big-O no es solo un modelo matemático abstracto, sino un factor que define directamente cómo operan y sobreviven las soluciones tecnológicas en el mundo real.